

Safeguard Effectiveness in Vibe Coding: An Empirical Evaluation of Test-Guided Repair and Regression in AI-Generated Full-Stack Web Applications

Can vibe-coded applications survive their own fixes? A deterministic, preregistered measurement of repair success, regression introduction, and their trajectories across repeated AI repair cycles — functional, security, accessibility, build, and persistence oracles included.

Sayed Ali Haider Shah (public developer identity: Cipher)

Independent Researcher · GitHub @iamcipherdev

This document freezes the study design, benchmark specifications, oracle suite, repair protocol, metric definitions, and analysis plan before data collection. No experimental results are reported. The accompanying repository implements every component and includes a harness-validation pilot whose data is excluded from all research claims.

RESEARCH PROTOCOL — STUDY IN PROGRESS

Version 1.0.1, frozen 2026-09-14. No experimental results are reported in this document.

Repository: <https://github.com/iamcipherdev/VibeGuardBench>

Abstract

Prompt-driven AI application builders—systems marketed for “vibe coding” of full-stack products—can produce convincing applications from a single specification. What happens next is less clear: when deterministic engineering feedback identifies concrete defects in such an application, can the same builder repair those defects without breaking previously passing functionality? This protocol defines an empirical study of that question. Three frozen full-stack application specifications (appointment booking, multi-role dashboard, ordering and inventory) are each generated three times by two commercial AI application builders, yielding 18 initial applications. A deterministic oracle suite—clean install and build, static type checking and linting, browser-level functional tests, API-driven authorization probes, static security analysis, secret and dependency scanning, axe-core accessibility checks, and restart-based persistence probes—measures each application. For every failing check, a machine-generated, fixed-template feedback message is delivered to the builder, after which the *entire* oracle suite re-executes, for up to three repair iterations. The study measures the repair success rate, the regression rate among previously passing checks, net quality change, and their trajectories across iterations and failure categories. The design, specifications, prompts, oracles, and analysis plan are frozen before data collection. This document reports no results: the experimental phase had not begun at the time of writing, and the only executed activity is a harness-validation pilot whose data is excluded from all research claims.

1 Introduction

The production of software by describing it in natural language has moved from research demos to commercial products. Prompt-driven builders such as Lovable, Bolt, and Replit generate complete web applications—frontend, backend, and persistence—from a written specification, and the practice of assembling software this way is widely known as vibe coding. Surveys of the area describe a fast-growing tool landscape and identify reliability and security of the generated products as open empirical problems [1].

Most evaluation effort concentrates on the first act: what does a builder produce from a specification? Benchmarks such as WebGen-Bench score initially generated websites with automated tests [2], FullStackBench scores generated solutions across full-stack task domains [3], and AppWorld verifies agent programs against a simulated world of applications [4]. This emphasis leaves a second, practically important act unmeasured: what happens when the generated application fails objective quality checks and the builder is asked to fix it. In classical automated program repair, the answer is sobering—patches that satisfy a test oracle frequently break behavior elsewhere, a phenomenon documented as test overfitting [14], patch overfitting [16], and preservation violations [15]. Whether modern AI application builders exhibit analogous behavior when repairing their own output, at the level of a whole application with functional, security, accessibility, build, and persistence concerns, has not, to our knowledge, been measured.

This protocol specifies such a measurement. The unit of study is an application instance: generated by a commercial builder, frozen, measured by a deterministic oracle suite, then repaired by the same builder under machine-generated feedback and measured again, up to three times. The design makes three methodological commitments. First, *deterministic measurement*: every check produces the same outcome given the same artifact; no LLM judge scores anything. Second, the *whole-suite rule*: after every repair, all checks re-execute, because regressions are only visible to checks that run. Third, *pre-freezing*: specifications, prompts, oracles, metric definitions, and the analysis plan were committed publicly before any experimental data exists.

Two boundaries of scope deserve emphasis at the outset. The study examines two commercial builders and three bounded application tasks; its conclusions concern those systems and tasks, not all AI coding tools. And because the builders’ internal models are not disclosed, the study compares *systems*, not models.

1.1 Contributions

This document contributes (i) a frozen, preregistration-style research protocol for measuring repair-and-regression dynamics of AI application builders; (ii) VibeGuardBench, an open benchmark implementation—three frozen application specifications with machine-checkable interface contracts, a 39-check deterministic oracle suite across eight categories, an experiment runner, a feedback generator, and a metrics and analysis pipeline; (iii) a related-work synthesis showing which parts of the question adjacent benchmarks leave unanswered; and (iv) a harness-validation pilot that demonstrates the full loop—generation, oracle execution, feedback, repair, and re-measurement—on real artifacts, with its data explicitly excluded from research claims.

2 Background and Related Work

2.1 Benchmarks for initial generation

WebGen-Bench evaluates LLM agents that generate websites from instructions, scoring them with automated functional tests [2]. A related benchmark, FullStackBench, covers about 4,400 tasks across full-stack domains and scores solutions in a sandbox [3]. AppWorld defines a controllable world of simulated apps with state-based verification of agent behavior [4]. DevBench and its successor evaluate repository-scale generation across the development lifecycle [5, 6], and Commit0 measures from-scratch library generation against given unit tests [7]. These systems establish that automated, test-based evaluation of generated software is feasible. They stop at initial generation: none of them feeds deterministic failure information back to the generating system and measures what breaks.

2.2 Repair against tests

SWE-bench established the evaluation of issue-resolving patches with repository test suites, separating FAIL_TO_PASS from PASS_TO_PASS outcomes [8]. A family of successors measures agentic repair [9], simplified non-agent pipelines [10], structure-aware fault localization [11], and the validity of the benchmark itself [12]. Notably, Wang et al. find that patches reported as solved can be incorrect beyond what the tests observe [13]. This line evaluates repair of human-authored repositories against unit tests; the artifact, the oracle, and the repair driver all differ from the setting studied here, in which a commercial builder repairs its own generated application against a multi-category QA suite.

2.3 Regression and overfitting in repair

That repaired software can be worse than its tests suggest is an old measurement. Smith et al. show that most automatically produced patches are plausible rather than semantically equivalent [14]. Poracle demonstrates that patches violate preservation conditions beyond the failing test [15]; Petke et al. quantify the magnitude of patch overfitting [16]; and Lou et al. show across two million candidate patches that regression suites reject a substantial share of APR output [17]. These studies concern classical search-based and semantic APR on Java and C. The present study measures the same phenomenon—repair that breaks passing behavior—in a modern setting: LLM-driven, builder-mediated repair of full-stack applications, with regressions defined across functional, security, accessibility, build, and persistence oracles rather than a unit-test suite alone.

2.4 Iterative modification and self-correction

CanItEdit evaluates instructional code editing, including successive edits where each must preserve prior behavior, and finds editing capability degrades with instruction complexity [18]. Self-Refine improves outputs with model-generated feedback [19], while Huang et al. show that intrinsic self-correction without external signals can degrade reasoning performance [20], and Reflexion uses environment signals across episodes [21]. Together this work suggests that the source of feedback matters: deterministic, tool-generated feedback, as fixed in our design, removes the self-judging confound that motivates the negative results.

2.5 Security of AI-generated code

Pearce et al. find that roughly 40% of Copilot’s completions in security-relevant scenarios contain vulnerabilities [22], and that LLMs can zero-shot repair a non-trivial share of vulnerabilities in C/C++ snippets [23]. Perry et al. show that developers assisted by an LLM write less secure code and are more confident in it [24]. Security measurement of generated *applications* in a repair loop, where the question is whether security failures get repaired and at what regression cost, remains open; our design includes static security analysis, secret detection, dependency auditing, and behavioral authorization probes per iteration, with scanner findings manually verified rather than taken at face value.

2.6 Gap

Our search of the venues and indexers listed in the repository’s search log found no empirical study combining commercial AI application builders, exported full-stack applications, a deterministic multi-category QA oracle suite, machine-generated repair feedback delivered back to the same builder, and whole-suite regression measurement across iterations. The closest works are WebGen-Bench (initial generation with tests, no repair loop) [2], the SWE-bench family (unit-test oracles on human repositories) [8], and the APR regression literature (pre-LLM tools) [17, 15]. Commercial comparisons of app builders exist in quantity as vendor and blog content, but not as reproducible empirical studies. We therefore claim a *coverage* gap, not a first: the study’s contribution is a careful measurement of a previously unmeasured combination, with primary weight on measurement validity.

3 Research Questions

- RQ1.** What proportion of objectively detected failures in AI-generated full-stack web applications become passing after the same builder receives deterministic, machine-generated feedback?
- RQ2.** How often do repair attempts introduce regressions, i.e., cause previously passing checks to fail, measured per failure category?
- RQ3.** Do repair success and regression rates differ across check categories (functionality, authentication/authorization, security, accessibility, build reliability, database persistence, type correctness, maintainability)?
- RQ4.** How do repair success and regression behavior change over up to three repeated repair cycles?
- RQ5 (exploratory).** Do the two studied builders differ in repairability and regression profiles? The planned sample supports interval-estimated description only.

Table 1: Design structure. Each cell contains one application per repetition; each application is a sequence of at most four measured suite executions (i0–i3).

Factor	Levels	Value
Task	3	APP-01 booking; APP-02 dashboard; APP-03 inventory
Builder	2	two commercial builders meeting §4 criteria
Repetition	3	independent generation runs per cell
Repair iterations	≤ 3	whole-suite re-measurement after each

RQ5 is deliberately demoted from confirmatory status: with 18 applications, a powered comparison between builders is not attainable, and reporting one as if it were would misstate the evidence.

4 Study Design

The experiment follows a factorial design with a longitudinal component at the check level: 3 application tasks $\times$ 2 builders $\times$ 3 independent repetitions, yielding 18 initial applications (iteration 0), each eligible for up to 3 repair iterations. Figure 1 summarizes the structure.

The scale is a deliberate trade-off. The statistical unit is the *check observation* (application $\times$ iteration $\times$ check); the frozen catalog contains 39 checks, and per-application applicability and cascade rules yield roughly 20–25 eligible checks per suite execution, hence roughly 1,500–2,000 eligible check observations across 72 suite executions and roughly 700–1,200 transition-eligible observations. At the application level the sample is small, so all application-level aggregates are reported with cluster-aware uncertainty, and the builder comparison (RQ5) is treated as exploratory. Larger grids were rejected because commercial builder subscriptions are metered, repair loops are operator-driven, and three repetitions already permit within-cell variance estimation under generation randomness.

5 Benchmark and Application Tasks

Three bounded applications were specified, each frozen in **benchmark/specifications/**: an appointment booking system (two roles, CRUD, search, admin panel), a multi-role SaaS dashboard (three roles, project–task hierarchy, role-scoped API), and an ordering/inventory application (order placement with stock decrement, manager-only inventory administration).

Each specification contains three components. First, numbered functional, security, accessibility, and persistence requirements written in prose precise enough to be checked. Second, a binding *interface contract*: exact HTTP routes, request and response JSON shapes, seed accounts, and DOM **data-testid** attributes. The contract is what makes deterministic cross-implementation testing possible; builders retain freedom over stack, architecture, and visual design, but generated applications that deviate from the contract fail the affected checks, and that failure is a measured outcome rather than a setup error. Third, an applicability map to the frozen check catalog.

The prescriptive contract trades ecological representativeness for measurement validity: freely prompted applications cannot be compared on deterministic oracles, because the oracles would differ per application. We record this trade-off as a threat in §12 and note that it biases *against* builders in the short term (contract violations appear as failures) while favoring measurement consistency across iterations.

Table 2: Oracle categories. Runtime categories are recorded as not applicable when the application cannot be started (cascade rule).

Category	Checks	Method
BUILD	5	fresh <code>npm ci/install</code> ; declared build; server start on \$PORT; root reachability; startup log hygiene
TYPE	1	<code>tsc --noEmit</code> for TypeScript projects
LINT	2	ESLint with a frozen framework-agnostic config; zero errors / zero warnings
FUNC	10–11	Playwright (Chromium) drives the interface contract: login, logout, create, list, render, edit, delete, validation, search, empty state, not-found, API rejection
AUTHZ	6	API probes across the role $\times$ route matrix, cross-owner writes, UI denial, logout invalidation
SEC	6	frozen Semgrep ruleset; secret-pattern scan; <code>npm audit</code> high/critical; debug-endpoint probes; cookie flags or non-localStorage tokens; credential storage
A11Y	4	axe-core (WCAG 2.1 A/AA subset) on login, primary list, and form pages; language and title
DB	4	create, update, delete survive full process restart; response shapes match the contract

6 Evaluation Oracles

The oracle suite contains 39 checks across eight categories (Table 2), each emitted with a frozen identifier from the check catalog. Suite execution is automated by a runner that installs, builds, launches, and probes the application, and records a schema-validated run record with raw tool output for every check.

Three rules govern interpretation. *Cascade*: when the application cannot be started, runtime checks are recorded as not applicable rather than failed, and the install/build failures carry the signal. *Flakiness policy*: environment-class failures (browser crash, localhost timeout) are retried once and logged; test-content failures are never retried; during the pilot, every oracle was executed twice back-to-back to detect non-deterministic checks. *Security caution*: scanner findings are candidate findings, not confirmed vulnerabilities. Every distinct security finding is manually inspected before analysis; a sensitivity analysis repeats the security results with findings judged false positives counted as passing. No CVSS scores are assigned and no exploitable-vulnerability claims are made from scanner output alone.

7 Repair Protocol

After each suite execution, a feedback generator renders one fixed-template message per failing check—deterministic text with the observed and expected values and a specification reference, e.g., “*AUTHZ-003 failed: request GET /api/admin/stats as role ‘patient’ returned 200; expected 403.*”—preceded by a preamble instructing the builder to repair without changing unrelated behavior. The operator delivers the feedback verbatim to the same builder session that produced the application, exports the resulting source, and the runner re-executes the *entire* suite. The whole-suite rule admits no exceptions: regressions are only observable to checks that run, and rerunning only the failing subset would undercount them by construction. Iteration 0 is archived once and never overwritten; repairs may stop early only when no checks are failing.

8 Analysis Plan

8.1 Metrics

Per application a and transition $k \rightarrow k+1$, with outcome function $X_{a,k}(\cdot)$ over checks, let $T(a, k) = \{c : X_{a,k}(c) = \text{fail}\}$ and $P(a, k) = \{c : X_{a,k}(c) = \text{pass}\}$, excluding environment-error and not-applicable checks from all counts:

$$\text{RSR}(a, k) = \frac{|\{c \in T(a, k) : X_{a,k+1}(c) = \text{pass}\}|}{|T(a, k)|}, \quad \text{RR}(a, k) = \frac{|\{c \in P(a, k) : X_{a,k+1}(c) = \text{fail}\}|}{|P(a, k)|}, \quad (1)$$

$$\text{NQC}(a, k) = |\text{repaired}| - |\text{regressed}|.$$

Raw counts accompany every rate; rates with fewer than five targets or five passing checks are flagged as small-denominator. Because checks within an application are not independent—a build failure cascades into every runtime check—rates are estimated with application-level clustering, and a sensitivity analysis repeats category-level comparisons after excluding cascading categories when BUILD fails at iteration k .

8.2 Statistical procedures

The plan is estimation-first. Descriptive trajectories per application and category come first; uncertainty is quantified by a cluster bootstrap over applications (2,000 replicates, percentile intervals), since checks are not independent units. If a model is used, two transition datasets are modeled separately—fail→pass outcomes for RQ1 and pass→fail outcomes for RQ2—with iteration $\times$ category fixed effects and random intercepts for application and check; a GLMM is fitted for a transition type only if at least ten applications contribute at least one event of that type, otherwise bootstrap-only inference is reported, and the choice is stated. Two pre-declared hypothesis families—category contrasts (RQ3, restricted to categories with at least eight eligible checks per application: functionality, authorization, security, persistence) and iteration contrasts (RQ4)—use Benjamini–Hochberg correction within family at $\alpha = .05$; smaller categories (build, type, lint, accessibility) are reported descriptively. Effect sizes are reported as risk differences with intervals. Engineering significance is discussed separately from statistical significance, and no importance claim rests on a p-value alone. All numbers in any future results section will be regenerated by repository scripts from the processed dataset; none are transcribed by hand. Two further points bound interpretation. Each repair transition bundles all failing checks into a single builder action, so per-check rates estimate check-level behavior under bundle feedback, not independent repair events. And the primary estimands use transition-eligible sets (checks whose outcome is defined at both iterations), with the protocol-literal fixed denominators reported as a conservative sensitivity and an explicit exclusion ledger; the protocol changelog records this amendment, made pre-data, alongside every other post-review change.

9 Status of Results

No experimental data has been collected at the time of this writing. The Results section of the eventual manuscript does not exist yet, and this document does not anticipate it. The protocol, specifications, prompts, oracles, and analysis plan are frozen pre-data; every post-freeze change is recorded in the repository changelog, and the changes below were all made before data collection, in response to adversarial review passes.

What *has* been executed is a harness-validation pilot, reported here only as evidence that the machinery works and only under that label. The pilot generated the APP-01 booking application with a non-study LLM system (GLM via a programmatic SDK; it is not Lovable, Bolt, or Replit, and the run is excluded from all research data), executed the full 39-check suite,

delivered machine-generated feedback for sixteen failures, executed a repair, re-ran the whole suite, and repeated once more. With the corrected harness, the measured trajectory was 19 passing / 16 failing at iteration 0; a repair that changed the code, repaired 0 of 16 failures, and regressed 1 of 19 passing checks at iteration 1 (the builder moved the session token into a cookie without the required security flags); and a correct repair of that same cookie-flags check at iteration 2 (repair success 1/17, regression rate 0/18). Persistent failures—missing interface attributes (which cascade into every UI-driven check), no root route, a stateless-token logout that does not invalidate sessions, a contract schema deviation, and one high and one critical dependency vulnerability—are plausible model behavior and demonstrate precisely the kind of dynamics the full study is designed to quantify. Fifteen harness defects found during the pilot and review passes (among them a lint configuration that produced spurious errors, an over-strict health check that misread redirects, orphaned server processes, an invalid scanner configuration, silent pass-through of dependency-audit failures, and cross-iteration state pollution) were fixed, and each fix is logged in the protocol changelog and the revision log. The pilot’s purpose was to find exactly these defects before they could contaminate study data.

10 Threats to Validity

Internal validity. Repair outcomes may depend on builder-side behavior we cannot observe or control: hidden repair logic, server-side model selection, and rate-limit-driven truncation. We log everything observable (platform version, tier, dates, exposed model disclosures) and report what cannot be pinned down as unknown rather than assuming it neutral. Feedback order and phrasing are frozen; the operator may not rephrase. State pollution across suite executions is mitigated by a logged database reset step. Scanner false positives are handled by manual verification plus a sensitivity analysis rather than by silent exclusion.

Construct validity. Check outcomes are proxies. “Passing” a functional check means the behavior holds under the contract’s fixed probes, not that the application is correct in general; a security check failing means a scanner or probe fired, not that an exploitable vulnerability exists. The regression construct is conservative by design: only previously passing checks that later fail count, and the whole-suite rule makes the measurement of that set explicit. Denominator choices are frozen before analysis, with raw counts always reported, because rate metrics with small or shifting denominators are easy to overstate.

External validity. Two builders, three tasks, and one operator bound generalization. The findings will concern the studied systems under the studied contracts; they will not license claims about all AI coding tools, about models, or about production software. The interface contracts constrain design freedom, which may suppress diversity in how builders fail; the counter-argument—that contract adherence is itself a property worth measuring—does not remove the threat, so we report it rather than argue it away.

Conclusion validity. With 18 applications and up to four measured iterations per application, the design supports cluster-bootstrapped interval estimates and cautious model-based inference; it does not support powered claims of builder equivalence or superiority. Category-level cells may be thin (some categories contain fewer than ten eligible checks per application); small-denominator flags and the sensitivity analyses exist for exactly this reason.

11 Reproducibility

The repository ships everything needed to re-run or extend the study: frozen specifications and prompts; the check catalog with per-check feedback templates; the oracle implementations

(Playwright suites, the runtime probe engine, the Semgrep ruleset, the ESLint configuration); the runner and feedback generator; JSON schemas for run records and check results; the ledger and analysis scripts; and the literature-search log with machine-readable citation-verification records. Every suite execution writes a schema-validated run record with environment versions, raw tool output, operator interventions, and retries. Generated application sources are archived per iteration, with any generated secrets redacted in public archives and redactions logged. A researcher starting from the repository README can execute the oracle suite against any exported application without access to the authors; the operator runbook documents the human steps for the builder-facing phases. Because the commercial builders are external services, exact reproduction of *builder behavior* is not possible—platform versions change—which is a property of the subject matter, documented rather than hidden.

12 Ethical Considerations

The study involves no human subjects and collects no personal data. Generated applications run only in a local pinned environment; nothing is deployed publicly. Security findings in generated applications are reported as study measurements; findings that appear to implicate a builder platform itself (rather than the generated application) would be disclosed to the vendor before publication. The study’s public artifacts may contain generated hard-coded credentials; these are redacted with an auditable log. The single-operator design creates a potential for unconscious bias in favor of the protocol; the preregistration-style freeze, logged operator interventions, and machine-generated feedback exist to keep that bias checkable rather than to deny its possibility.

13 Limitations

Beyond the threats above, three limitations are inherent to the design. First, iteration 0 already includes the frozen specification, so the study measures repair of specification-conditioned applications, not of arbitrary prompts. Second, the maximum of three repair iterations bounds trajectory claims; behavior at iteration five or ten is out of scope. Third, the builder market moves quickly: platform versions will drift during data collection, and the run records are the only reliable account of which version produced which artifact.

14 Future Work

Three extensions follow naturally. A larger builder panel with metered API access would convert RQ5 into a powered comparison. Feedback-ablation arms—bare failure messages versus located diagnostics versus specification excerpts—would test which feedback components drive repair, replacing the fixed feedback of this protocol. And the same oracle suite applied to human-authored starter codebases would separate “repair of AI-generated code” from “repair of this kind of application,” a confound this within-generated design cannot address.

15 Conclusion

Whether AI application builders can repair their own output without breaking what already works is a concrete, measurable, and practically important question, and one that existing benchmarks do not answer. This protocol fixes the specifications, oracles, feedback mechanism, metrics, and analysis plan needed to answer it for two commercial builders and three bounded applications, before any data is collected. The repository accompanying this document is the protocol: running its harness against exported applications is the study.

References

- [1] Y. Ge, L. Mei, Z. Duan, T. Li, Y. Zheng, et al. A Survey of Vibe Coding with Large Language Models. arXiv:2510.12399, 2025.
- [2] Z. Lu, Y. Yang, H. Ren, H. Hou, H. Xiao, K. Wang, W. Shi, A. Zhou, et al. WebGen-Bench: Evaluating LLMs on Generating Interactive and Functional Websites from Scratch. Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2505.03733.
- [3] ByteDance-Seed Foundation Code Team. FullStack Bench: Evaluating LLMs as Full Stack Coders. arXiv:2412.00535, 2024.
- [4] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. Proceedings of ACL 2024. DOI 10.18653/v1/2024.acl-long.850.
- [5] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li, S. Yao, C. Qian. Prompting Large Language Models to Tackle the Full Software Development Lifecycle: A Case Study (benchmark artifact: DevBench). arXiv:2403.08604, 2024.
- [6] A. Kumarappan, P. A. Golnari, W. Wen, X. Liu, G. Ryan, Y. Sun, S. Fu. DevBench: A Realistic, Developer-Informed Benchmark for Code Generation Models. arXiv:2601.11895, 2026.
- [7] W. Zhao, N. Jiang, C. Y. Lee, J. Chiu, C. Cardie, et al. Commit0: Library Generation from Scratch. arXiv:2412.01769, 2024.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? International Conference on Learning Representations (ICLR), 2024. arXiv:2310.06770.
- [9] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [10] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.
- [11] Y. Zhang, H. Ruan, Z. Fan, A. Roychoudhury. AutoCodeRover: Autonomous Program Improvement. Proceedings of ISSTA 2024. DOI 10.1145/3650212.3680384.
- [12] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin, S. Wang. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992, 2024.
- [13] Y. Wang, M. Pradel, Z. Liu. Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. Proceedings of ICSE 2026. DOI 10.1145/3744916.3764576.
- [14] E. K. Smith, E. T. Barr, C. Le Goues, Y. Brun. Is the cure worse than the disease? Overfitting in automated program repair. Proceedings of ESEC/FSE 2015. DOI 10.1145/2786805.2786825.
- [15] E. Ismayilzada, M. M. ur Rahman, D. Kim, J. Yi. Poracle: Testing Patches under Preservation Conditions to Combat the Overfitting Problem of Automated Program Repair. ACM Transactions on Software Engineering and Methodology, 33(2), 2023. DOI 10.1145/3625293.
- [16] J. Petke, M. Martínez, M. Kechagia, A. Aleti, F. Sarro. The Patch Overfitting Problem in Automated Program Repair: Practical Magnitude and a Baseline. Companion Proceedings of FSE 2024, pp. 452–456. DOI 10.1145/3663529.3663776.
- [17] Y. Lou, J. Yang, S. Benton, D. Hao, L. Tan, Z. Chen, L. Zhang, L. Zhang. When Automated Program Repair Meets Regression Testing—An Extensive Study on Two Million Patches. ACM Transactions on Software Engineering and Methodology, 33, 2024. DOI 10.1145/3672450.
- [18] F. Cassano, L. Li, A. Sethi, N. Shinn, A. Brennan-Jones, et al. Can It Edit? Evaluating the Ability of Large Language Models to Follow Code Editing Instructions. arXiv:2312.12450, 2023.

- [19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, et al. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [20] J. Huang, X. Chen, S. Mishra, H. Zheng, A. W. Yu, X. Song, D. Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. *arXiv:2310.01798*, 2023 (ICLR 2024).
- [21] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [22] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. *Proceedings of IEEE S&P 2022*. DOI 10.1109/SP46214.2022.9833571.
- [23] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt. Examining Zero-Shot Vulnerability Repair with Large Language Models. *Proceedings of IEEE S&P 2023*. DOI 10.1109/SP46215.2023.10179324.
- [24] N. Perry, M. Srivastava, D. Kumar, D. Boneh. Do Users Write More Insecure Code with AI Assistants? *Proceedings of ACM CCS 2023*. DOI 10.1145/3576915.3623157.

Tools and standards

- [25] Semgrep. <https://semgrep.org>. Accessed 2026-09-14.
- [26] Playwright. Microsoft. <https://playwright.dev>. Accessed 2026-09-14.
- [27] axe-core. Deque Labs. <https://github.com/dequelabs/axe-core>. Accessed 2026-09-14.
- [28] W3C. Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>.
- [29] OWASP Top 10 (2021). <https://owasp.org/Top10/>.